Distributed Candy Crush

A Load-Balanced Web Game using Java Socket Programming

Nanditha Choudary Doddapuneni
23MCCE36

School of Computer and Information Sciences
University of Hyderabad

Agenda

1. The App
2. Original Architecture
3. Paradigms and Algorithms
4. My Simplified Version
5. Distributed Algorithms
6. Mapping
7. Demo
8. Conclusion

App Chosen: Candy Crush Saga

- Match-3 puzzle game by King (Activision Blizzard, now Microsoft)
- Launched on Facebook in April 2012 and on mobile in November 2012
- About 245 million monthly active users (2021)
- Needs a scalable, highly available distributed backend

Candy Crush Architecture (from King's talk)

- Sharded MySQL, with the **user ID** as the shard key
- Game state stored as a JSON blob; access is almost purely key-value
- No joins inside shards; one central shard catalogue
- Master/slave pairs with **manual** failover (KISS)
- Memcached in front of the database
- Offline play: client state overrides the server on reconnect
- Backend today: Java services on Google Cloud

Source: Wennmark, *MySQL@King*, Oracle OpenWorld 2014; King job postings.

Scale at the Time of the Talk (2014)

- User shards over 50 TB on 100+ MySQL instances
- More than 500,000 queries per second; read:write about 10:1
- Memcached handled about 2 million queries per second
- Operations team of only 4 people
- Responses over 10 ms trigger alarms and throttling
- "Not a bank": strict consistency not required

How Sharding Works in the Original

1. Request arrives for User 100
2. Sharding API looks up the shard in the catalogue (Shard 1)
3. Data is fetched from Shard 1
4. Result is returned to the requester

Scale up (bigger machine) vs scale out (shard by function, columns or rows). King scaled out.

Paradigms in the Original App

- Client-server
- Thick client / thin server (offline play)
- Key-value data access
- Service-oriented backend
- Publish-subscribe for analytics (commonly used, not confirmed)

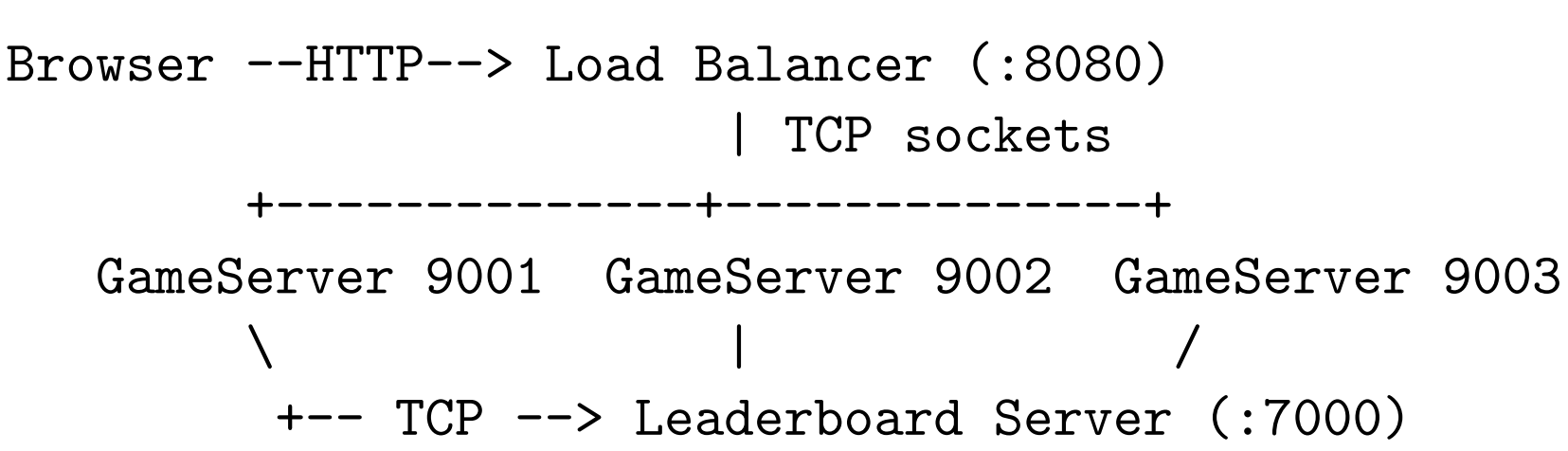
Algorithms in the Original App

- Sharding by user ID: scalability
- Master/slave replication: availability
- Manual failover with home-built tools
- Memcached caching: low latency
- Throttling on slow responses: overload protection
- Client-state override on reconnect: simple conflict rule
- Load balancing, CDN, auto-scaling: commonly used, not confirmed

Project Overview

- Browser-based Candy Crush with a 6x6 board
- 3 game servers behind a load balancer
- Shared leaderboard server with mutual exclusion
- Server components communicate over TCP sockets (Java)
- Only browser-to-balancer traffic uses HTTP

System Architecture



Components

Component	**Role**	**Port**
Load Balancer	Website, routing, heartbeats	8080
Game Server (x3)	Board and cascade logic	9001–9003
Leaderboard Server	Shared scores, lock	7000

Design Choice: Thin Client, Thick Server

- Original keeps logic on the client; mine keeps it on the servers
- Every move produces visible socket communication
- Makes the distributed algorithms observable
- Trade-off: no offline play
- Confirmed acceptable by the course instructor

Least-Connections Load Balancing

- New player goes to the live server with the fewest players
- Spreads load so no server is overloaded
- Session affinity: a player stays on one server

Heartbeat Failure Detection and Failover

- Balancer sends PING to every game server every 2 seconds
- No PONG: server marked DOWN, receives no new players
- If a player's server dies, they move to a live server
- Score is kept because it lives in the Leaderboard Server
- King's failover is manual; mine detects failure automatically

Centralized Mutual Exclusion

- Leaderboard is a plain HashMap shared by all game servers
- Every SET / GET / REMOVE / ALL acquires an explicit lock
- Only one request is in the critical section at a time
- No lost updates or race conditions
- Leaderboard Server acts as the central coordinator

Other Mechanisms

- Thread-per-request concurrent servers
- Timeout-based cleanup: inactive players removed after 15 s
- Cascade resolution: find, clear, drop, refill, repeat until stable
- Constrained random board: no matches at the start

Original App vs My Project

Original	**Mine**
Shards by user ID, 100+ instances	One leaderboard server, no sharding
Master/slave, manual failover	Automatic failure detection and reassignment
Memcached cache	No cache
Key-value state as JSON	Simple text protocol, score as value
Client logic, offline play	Server logic (deliberate)
Load balancing over servers	Least-connections balancer
Consistent shared state	Lock-based mutual exclusion

Live Demo

1. Start all 5 processes
2. Join with 3 players and see them spread across servers
3. Make a move and watch the cascade and leaderboard
4. Kill a game server, watch it turn DOWN, see failover
5. Restart the server and see it return

Lessons from King Applied to My Design

- Keep the design simple (KISS): small, separate components
- Keep durable state (scores) outside the servers that can fail
- Key-value style access makes sharding practical
- Offline-friendly clients reduce the need for strict consistency

Limitations and Future Work

- Load balancer and leaderboard server are single points of failure
- No persistence (data in memory)
- Future: shard the leaderboard by player name with a lookup catalogue
- Future: add a Memcached-style cache and replicate the coordinator

Conclusion

- A working distributed web application over Java sockets
- Load balancing, failure detection, failover, mutual exclusion
- Design choices linked back to King's published architecture

References

- Wennmark, T. *MySQL@King*, Oracle OpenWorld 2014
- King careers: Candy Crush backend and platform postings
- "Candy Crush Saga", Wikipedia
- Internet Museum (Sweden), *Candy Crush comes to Facebook*
- Tanenbaum & van Steen, *Distributed Systems*
- Coulouris et al., *Distributed Systems: Concepts and Design*